

AIM

Write a C program to demonstrate basic system calls in Linux.

SOFTWARE REQUIRED

- A Linux operating system
- GCC (GNU Compiler Collection) for compiling C programs
- Terminal or command line interface

THEORY

System calls are fundamental components of the Linux operating system that allow user-space programs to request services from the kernel. These calls serve as a bridge between software applications and the hardware. Common system calls include:

- write(): Used to write data to a file descriptor.
- read(): Used to read data from a file descriptor.
- open(): Used to open a file.
- close(): Used to close a file descriptor.
- fork(): Used to create a new process by duplicating the calling process.
- exec(): Used to execute a program within a process.
- exit(): Used to terminate a process.

In this experiment, we will focus on demonstrating the write(), open(), and close() system calls with a simple C program.

PROCEDURE

1. Create a C Program:

Write a C program that uses the open(), write(), and close() system calls.

2. #include <stdio.h>

```
#include <unistd.h>    // fork, getpid, read, write, close
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>  // wait
```

```
#include <fcntl.h>     // open
```

```
int main() {
```

```
    // 1. write() system call - write to standard output
```

```
    write(1, "Hello from write() syscall!\n", 29);
```

```
    // 2. getpid() system call - get process ID
```

```

pid_t pid = getpid();

printf("My process ID (getpid): %d\n", pid);

// 3. fork() system call - create a new process

pid_t child = fork();

if (child == 0) {

    // This runs in the CHILD process

    printf("Child process running, PID = %d\n", getpid());

} else {

    // This runs in the PARENT process

    wait(NULL); // wait() syscall - wait for child to finish

    printf("Parent process running, PID = %d, Child PID = %d\n", getpid(), child);

}

// 4. open(), write(), close() system calls - file handling

int fd = open("demo_output.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);

if (fd != -1) {

    write(fd, "This line was written using open()/write()/close() syscalls.\n", 62);

    close(fd);

    printf("File written using open/write/close syscalls.\n");

}

return 0;

}

```

3. Compile the Program:

Use GCC to compile the C program.

4. gcc -o syscalls_demo syscalls_demo.c

5. Execute the Program:

Run the compiled program to observe the output.

6. ./syscalls_demo

7. Verify the Output:

Check the content of output.txt to verify the data written by the program.

8. cat output.txt

OUTPUT

Upon executing the program, the console should display:

Hello from write() syscall!

My process ID (getpid): <PID>

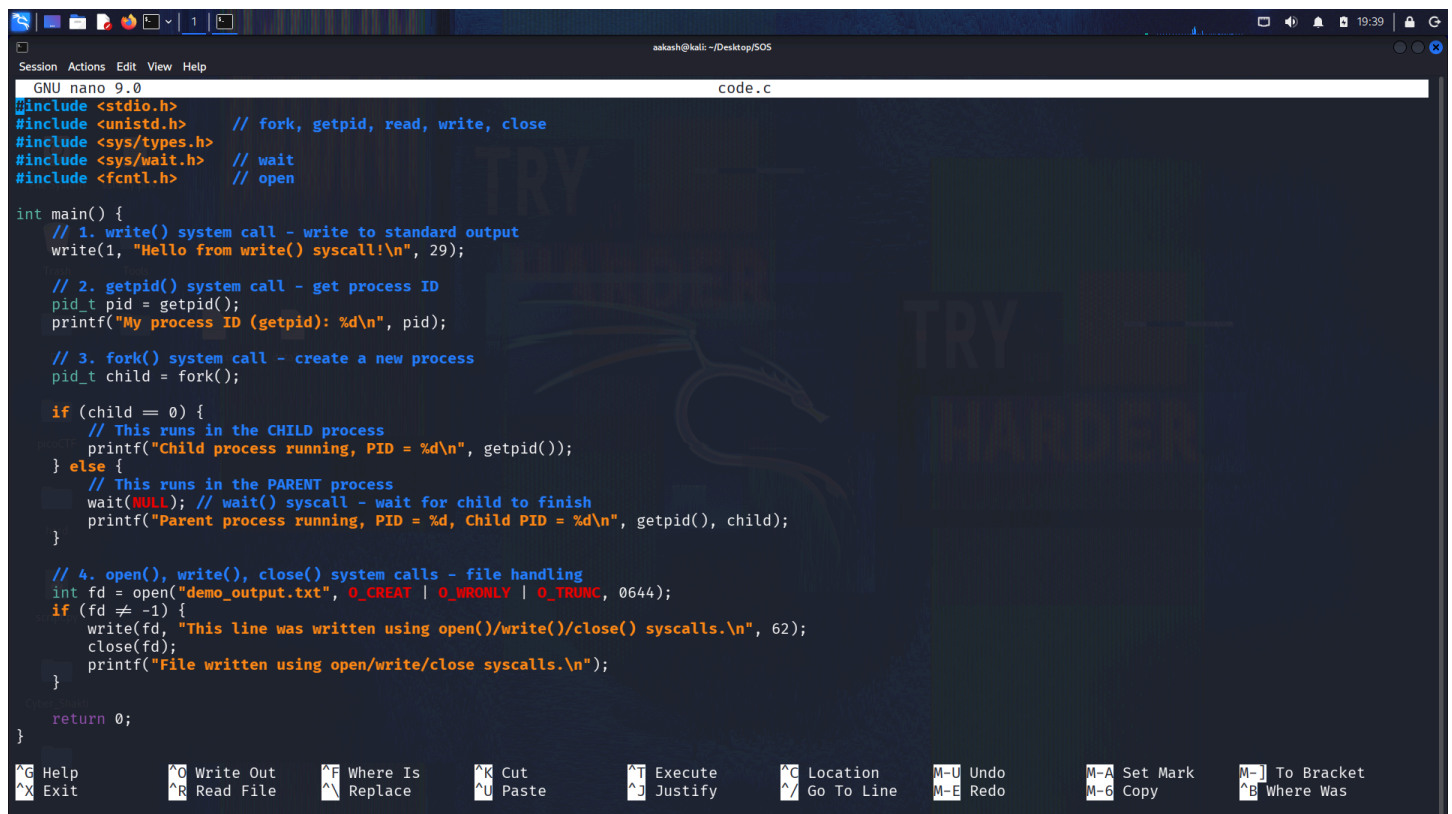
Child process running, PID = <PID>

File written using open/write/close syscalls.

Parent process running, PID = <PID>, Child PID = <PID>

File written using open/write/close syscalls.

SCREENSHOT



```
GNU nano 9.0 code.c
#include <stdio.h>
#include <unistd.h> // fork, getpid, read, write, close
#include <sys/types.h>
#include <sys/wait.h> // wait
#include <fcntl.h> // open

int main() {
    // 1. write() system call - write to standard output
    write(1, "Hello from write() syscall!\n", 29);

    // 2. getpid() system call - get process ID
    pid_t pid = getpid();
    printf("My process ID (getpid): %d\n", pid);

    // 3. fork() system call - create a new process
    pid_t child = fork();

    if (child == 0) {
        // This runs in the CHILD process
        printf("Child process running, PID = %d\n", getpid());
    } else {
        // This runs in the PARENT process
        wait(NULL); // wait() syscall - wait for child to finish
        printf("Parent process running, PID = %d, Child PID = %d\n", getpid(), child);
    }

    // 4. open(), write(), close() system calls - file handling
    int fd = open("demo_output.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd != -1) {
        write(fd, "This line was written using open()/write()/close() syscalls.\n", 62);
        close(fd);
        printf("File written using open/write/close syscalls.\n");
    }

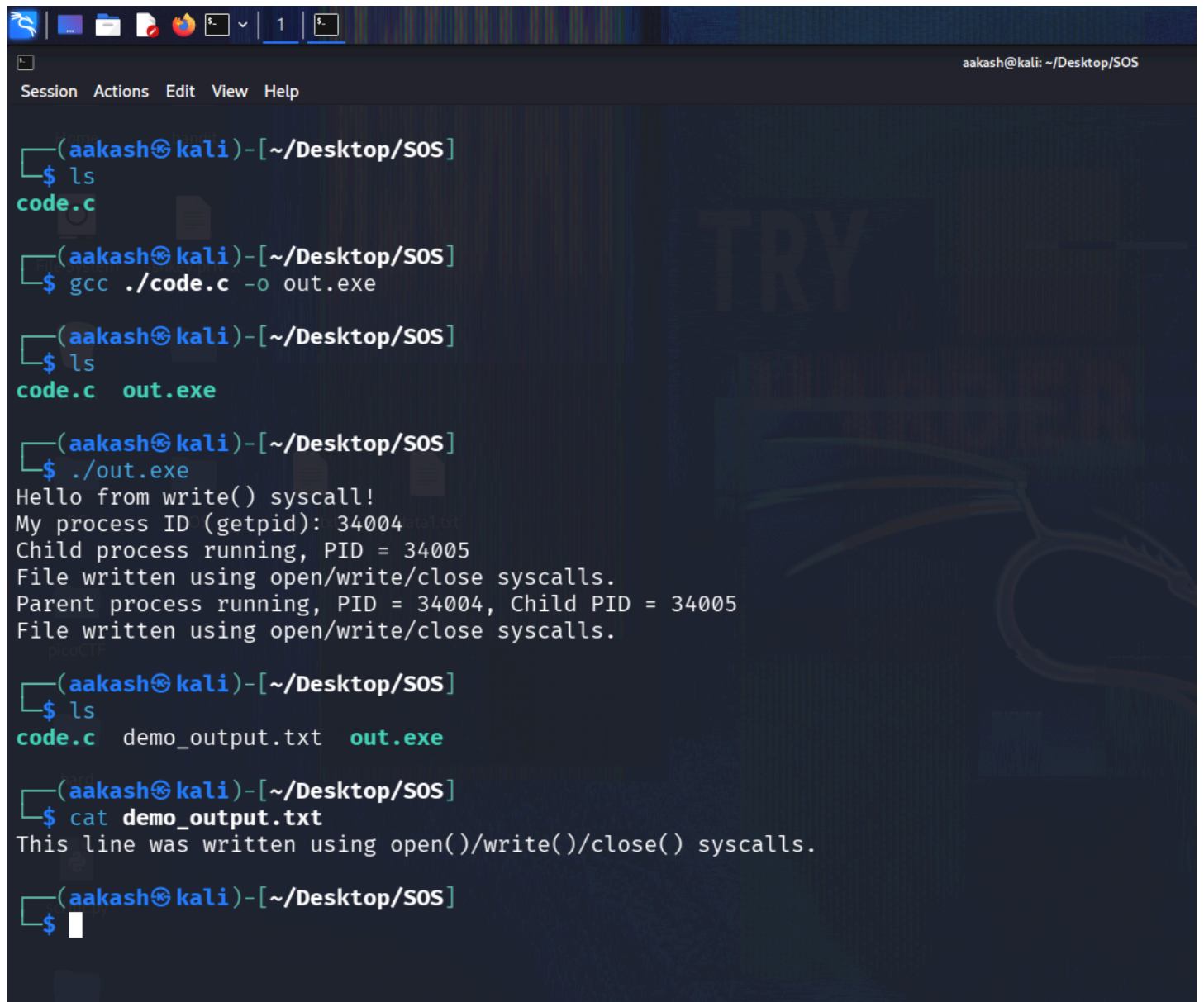
    return 0;
}
```

Session Actions Edit View Help

GNU nano 9.0 code.c

aakash@kali: ~/Desktop/SOS 19:39

^G Help ^O Write Out ^F Where Is ^X Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket
^X Exit ^R Read File ^_ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo M-G Copy ^B Where Was



```
(aakash@kali)-[~/Desktop/SOS]
$ ls
code.c

(aakash@kali)-[~/Desktop/SOS]
$ gcc ./code.c -o out.exe

(aakash@kali)-[~/Desktop/SOS]
$ ls
code.c  out.exe

(aakash@kali)-[~/Desktop/SOS]
$ ./out.exe
Hello from write() syscall!
My process ID (getpid): 34004
Child process running, PID = 34005
File written using open/write/close syscalls.
Parent process running, PID = 34004, Child PID = 34005
File written using open/write/close syscalls.

(aakash@kali)-[~/Desktop/SOS]
$ ls
code.c  demo_output.txt  out.exe

(aakash@kali)-[~/Desktop/SOS]
$ cat demo_output.txt
This line was written using open()/write()/close() syscalls.

(aakash@kali)-[~/Desktop/SOS]
$
```

CONCLUSION

This experiment successfully demonstrated the use of basic system calls in a Linux environment using C. By employing the `open()`, `write()`, and `close()` system calls, we were able to interact with the file system, illustrating how user-space programs can request services from the kernel. This foundational understanding is crucial for developing more advanced system-level applications.